

Agent Chat Error Guide

Previous code, fixes, locations, and how to continue the project

Current status: The OpenRouter key was tested directly and returned HTTP 200. The authenticated chat endpoint also returned agent works.

1. Request flow

Browser

```
-> Vite proxy (127.0.0.1:4000)
-> Express POST /api/agent/chat
-> create/reuse Mastra memory thread
-> official @openrouter/sdk
-> OpenRouter /api/v1/chat/completions
-> Server-Sent Events (SSE) back to browser
```

2. Problems in the previous code

#	Problem	Where	Effect
1	Proxy used localhost	frontend/vite.config.js	Windows IPv4/IPv6 resolution could reset the proxy connection.
2	SSE did not detect disconnected clients	backend/src/routes/agent.routes.js	Backend could write to a closed connection and cause ECONNRESET.
3	OpenAI key/provider was invalid	backend/src/services/agent.service.js	OpenAI returned HTTP 401, so the agent generated no answer.
4	Wrong OpenRouter transport was tried	backend/src/services/agent.service.js	The OpenAI-compatible attempt used the Responses endpoint; OpenRouter returned User not found.
5	New thread was recalled before creation	backend/src/services/agent.service.js	First message failed with No thread found.

6	Stale environment variable won over .env	backend/src/index.js	Running server used an old API key even though the key in .env was valid.
7	Provider error could look like success	backend/src/services/agent.service.js	The stream could finish without a token because the error event was ignored.

3. Important before/after changes

Proxy target

```
// Previous
target: "http://localhost:4000"
```

```
// Current
target: "http://127.0.0.1:4000",
timeout: 600000,
proxyTimeout: 600000
```

Environment loading

```
// Previous
config({
  path: resolve(fileURLToPath(new URL("../.env", import.meta.url))),
});
```

```
// Current
config({
  path: resolve(fileURLToPath(new URL("../.env", import.meta.url))),
  override: true,
});
```

The override is important when the terminal already contains an old OPENROUTER_API_KEY.

OpenRouter client

```
// Previous provider approach
createOpenAI({
  apiKey,
  baseUrl: "https://openrouter.ai/api/v1",
});
```

```
// Current official SDK
import { OpenRouter } from "@openrouter/sdk";

const client = new OpenRouter({
  apiKey: process.env.OPENROUTER_API_KEY,
  httpReferer: process.env.OPENROUTER_HTTP_REFERER,
  appTitle: process.env.OPENROUTER_TITLE,
});

const completion = await client.chat.send({
  chatRequest: {
    model: process.env.AI_MODEL,
    messages,
  },
});
```

New thread creation

```
const existingThread = await memory.getThreadById({
  threadId: input.threadId,
  resourceId: input.userId,
});

if (!existingThread) {
  await memory.createThread({
    threadId: input.threadId,
    resourceId: input.userId,
    title: "",
  });
}
```

4. Environment

```
OPENROUTER_API_KEY=your_valid_key
AI_MODEL=openai/gpt-4o-mini
OPENROUTER_TITLE=Agentic Calendar Assistant
```

Never commit backend/.env. Any key pasted into chat, source code, or a public repository must be revoked and replaced.

5. How to run

1. In agentic-calendar-assistant/backend, run `npm run dev`.
2. Confirm `http://127.0.0.1:4000/health` returns status ok.
3. In a second terminal, open agentic-calendar-assistant/frontend and run `npm run dev`.
4. Open the shown frontend URL, sign in, and send a message.

6. Checks

```
cd backend
npm test

cd ../frontend
npm run build
```

Expected backend result: **23 tests passed**.

7. Successful response

```
data: {"type":"started","message":"Agent is planning"}

data: {"type":"token","token":"agent works"}

data: {"type":"completed","message":"done"}
```

8. If an error returns

- **ECONNRESET**: confirm backend is running on port 4000 and restart Vite.
- **OpenRouter rejected key**: revoke the exposed key, create a new key, update .env, and restart the backend.
- **No thread found**: confirm the latest backend code is running and not an old process.
- **Empty answer**: check the backend terminal and confirm the model is available.

Generated for the Agentic Calendar Assistant after the OpenRouter SDK, SSE, thread initialization, proxy, and environment-loading fixes.